

ComicCraft - AI Comic Story Creator using Gemini Models

ComicCraft is a web-based application that uses AI to generate personalized comic book stories and illustrations from user-provided prompts. Built with **FastAPI** and powered by **Google Gemini** models and **Stable Diffusion**, it automates the creation of storylines, dialogues, and comic-style imagery, then lets users preview and export the result as a PDF.

Features

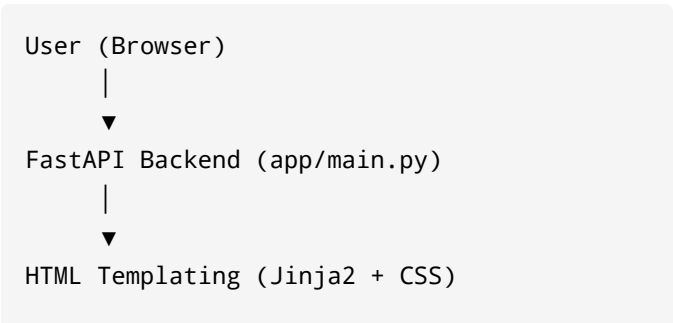
- Generate a full comic from a single story prompt
- Customize main character name, setting, tone, and art style
- AI-generated panel-by-panel storyline with narration and dialogue
- AI-generated comic-style illustrations for each panel
- In-browser comic preview
- One-click export to a professionally formatted PDF

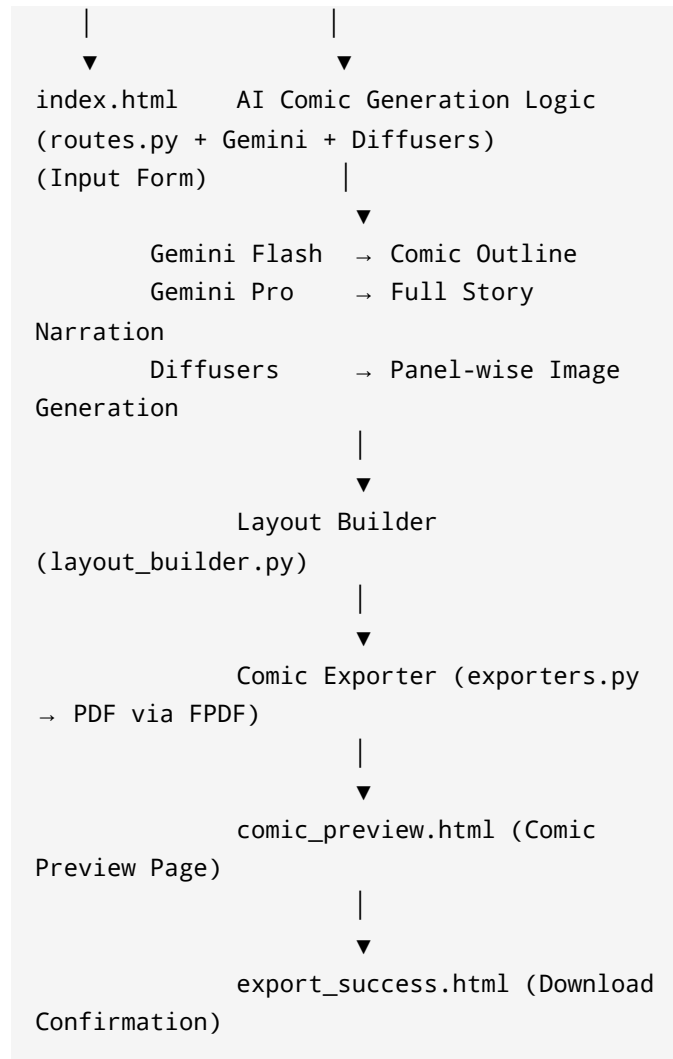
- Both HTML form and JSON API endpoints for comic generation

Tech Stack

Layer	Technology
Backend	FastAPI, Uvicorn
Frontend	HTML, CSS, Jinja2
Story generation	Google Gemini Flash (gemini-1.5-flash) & Gemini Pro (gemini-1.5-pro)
Image generation	Stable Diffusion (runwayml/stable-diffusion-v1-5) via Hugging Face Diffusers
PDF export	FPDF

Architecture





Project Structure

```
comiccraft/  
|
```

```

├── app/
│   ├── main.py
│   ├── routes.py
│   ├── gemini_flash.py
│   ├── gemini_pro.py
│   ├── image_generator.py
│   ├── layout_builder.py
│   └── exporters.py
│
├── templates/
│   ├── index.html
│   ├── comic_preview.html
│   └── export_success.html
│
├── static/
│   ├── panels/      # Generated panel
images
│   ├── exports/     # Exported comic PDFs
│   └── fonts/       # Fonts for PDF
generation
│
├── .env             # Environment
variables (API keys)
└── requirements.txt # Python
dependencies

```

Prerequisites

- FastAPI Framework Knowledge
- Uvicorn ASGI Server
- Python Programming Proficiency

- HTML, CSS, and Jinja2 Template Skills
- Google Gemini API Familiarity
- Hugging Face Diffusers & Stable Diffusion
- PyTorch Basics
- PDF Generation with FPDF
- Version Control with Git
- Python Package & Environment Setup (pip, virtualenv, or conda)

Setup

1. Create and activate a virtual environment

```
python -m venv comiccraft-env
comiccraft-env\Scripts\activate # Windows
source comiccraft-env/bin/activate #
macOS/Linux
```

2. Install dependencies

```
pip install fastapi uvicorn jinja2 python-
multipart google-generativeai diffusers
transformers fpdf Pillow accelerate
```

3. Configure environment variables

Create a `.env` file in the project root:

```
GEMINI_API_KEY=your_gemini_api_key_here
HF_API_KEY=your-huggingface-api-key-here
```

4. Run the server

```
uvicorn app.main:app --reload
```

5. Access the app

- Web app: <http://127.0.0.1:8000>
- API docs (Swagger): <http://127.0.0.1:8000/docs>

API Routes

Route	Method	Description
/	GET	Loads the homepage (index.html) for story input
/generate	POST	Handles form submission, runs the full AI pipeline, returns the comic preview page

Route	Method	Description
<code>/generate-comic/json</code>	POST	JSON API equivalent of <code>/generate</code> ; returns layout data and PDF path
<code>/export-success</code>	GET	Displays a success confirmation page after PDF download
<code>/test-image</code>	GET	Developer utility to test image generation from a direct prompt

How It Works

1. **Input** – User submits a story prompt, character name, setting, tone, and art style via `index.html` or a JSON payload.
2. **Outline generation** – `generate_outline()` (Gemini Flash) produces a structured 5-panel outline (panel number, title, scene description, image prompt).
3. **Story generation** – `generate_story()` (Gemini Pro) expands the outline into full narration and character dialogue.
4. **Image generation** – `generate_image()` (Stable Diffusion) creates an illustration for each panel

and saves it to `static/panels/`.

5. **Layout building** – `build_comic_layout()` merges images, story text, and outline data into a structured panel list.
6. **PDF export** – `save_pdf()` compiles the layout into a timestamped, multi-page PDF saved to `static/exports/`.
7. **Preview & download** – The comic is rendered panel-by-panel in `comic_preview.html`, with a download button that triggers the PDF export and redirects to `export_success.html`.

Development Milestones

1. **Model Selection & Architecture** – Chose Gemini Flash/Pro and Stable Diffusion; defined the frontend/backend/AI architecture; set up the dev environment.
2. **Core Functionalities** – Built outline generation, story generation, image generation, layout building, and PDF export; implemented the FastAPI backend for routing and input handling.
3. **routes.py Development** – Wired up all core routes and integrated the AI workflow end-to-end with error handling.
4. **Frontend Development** – Built `index.html`, `comic_preview.html`, and

`export_success.html` with responsive CSS and dynamic Jinja2 rendering.

5. **Deployment** – Configured the local environment, environment variables, and verified the full workflow via Uvicorn.

Notes

- The current version supports single comic generation without user accounts.
- The modular architecture supports future enhancements such as user profiles, comic libraries, and multi-page story arcs.